Principy objektového programování, agregace a kompozice objektů

Evoluce programování k objektově orientovanému programování

Strojový kód

Je to posloupnost strojových instrukcí prováděných procesorem počítače, která je zapsána pomocí posloupnosti číselných kódů těchto strojových instrukcí.

Instrukce má většinou dvě základní části - kód operace, který udává, co se má udělat, a adresy (někdy i více adres), který říká, s jakými daty se má operace provést.

Instrukce jsou zapsány čísly ve dvojkové nebo šestnáctkové soustavě

Nedají se v něm pojmenovávat proměnné.

Nestrukturované paradigma

Nestrukturovaný přístup je podobný assemblerům, jedná se o soubor instrukcí, který se vykonává odshora dolů.

Zdrojový kód již nebyl závislý na hardwaru a byl lépe čitelný pro člověka, přístup na nějakou dobu umožnil vytvářet komplexnější programy.

Strukturované paradigma

Programujeme pomocí cyklů a větvení.

Program lze rozložit do funkcí.

Celý program se potom skládá z nezapouzdřených bloků kódu a špatně se udržuje.

Objektově orientované programování

Jedná se o filozofii a způsob myšlení, designu a implementace, kde klademe důraz na znuvupoužitelnost.

Je to nový pohled na funkci programu a komunikaci mezi jeho jednotlivými částmi.

Jeho způsob fungování se silně odráží od teorie objektů.

Poskládání programu z komponent je výhodnější a levnější.

Fungování

Def: člověk se odpoutá od toho, jak program vidí počítač a píše program spíše z pohledu svého. To má značné výhody už jen v tom, že je to pro nás přirozenější a přehlednější.

Základní jednotkou je objekt, který odpovídá nějakému objektu z reálného světa.

Objektově orientované programování

Teorie objektů

Def z wiki:

Existuje obecná teorie, která se zabývá objekty, jejich vlastnostmi a vším k tomu přidruženým. Základním pojmem této teorie je třída, což je pojem ze všech asi nejobecnější - používáme ho ke klasifikaci procesu, jak uspořádat informace, ze kterých chceme náš výsledek složit, do smysluplných struktur. Podpojmem třídy je objekt, který využije předpis stvořený pomocí třídy a stane se takzvanou instancí třídy. Objekt má dva základní "smysly svojí existence" - pamatovat si svůj stav pomocí atributů a poskytovat své metody k tomu, aby s objektem šlo zvnějšku pracovat. Když použijeme objekt, zajímá nás, co se stane a jaký bude výsledek, ale v momentě použití už nás nezajímá, jak přesně se to stalo (za předpokladu, že nejste člověk pověřený testováním objektu).

Třída

Třída je vzor, podle kterého se objekty vytváří. Definuje jejich vlastnosti a schopnosti.

Objekt, který se vytvoří podle třídy, se nazývá instance. Instance mají stejné rozhraní jako třída, podle které se vytváří, ale navzájem se liší svými daty (atributy).

Atributy

Atributy objektu jsou vlastnosti neboli data, která uchovává.

Např: Clovek – Jmeno, Prijmeni.

Jedná se o prosté proměnné.

Metody

Metody jsou schopnostmi, které umí objekt vykonávat. Metody mohou mít parametry a mohou také vracet nějakou hodnotu.

Objektově orientované programování je založeno na 3 základech

- Zapouzdření
- Dědičnost
- Polymorfismus

Zapouzdření

Zapouzdření umožňuje skrýt některé metody a atributy tak, aby zůstaly použitelné jen pro třídu zevnitř.

Má několik úkolů,

- public – Viditelnost pro rozhraní třídy (instance).
- protected – Viditelnost pro objekt samotný a pro objekty odděděné od této třídy.
- private - Vlastnost vidí jen objekt samotný.

Dědičnost

Jedná se o ustanovení vztahu mezi objekty. V třídové dědičnosti, kde jsou objekty definované třídami, mohou třídy zdědit atributy a chování od předem existujících tříd, které se nazývají rodičovské třídy, základní třídy nebo super třídy. Výsledné třídy jsou nazývány odvozené třídy, podtřídy nebo potomek třídy

Je to jedna z hlavních výhod OOP, protože to ušetří psaní stejného kódu pořád dokola, pro nové třídy = slouží k tvoření nových datových struktur na základě starých.

V C# může jedna třída dědit pouze jednou, implementovat vícekrát.

Polymorfismus

Polymorfismus umožňuje používat jednotné rozhraní pro práci s různými typy objektů.

Příklad: Zdědění interface třídy, od které dědí ostatní (nebo interface samotného)

Polymorfismus umožňuje přepsat si zděděnou metodu u každé podtřídy tak, aby dělala, co třída požaduje. Rozhraní zůstane zachováno a nebude se nijak lišit v případě téhle metody.

podstatou polymorfismu je tedy metoda nebo metody, které mají všichni potomci definované se stejnou hlavičkou, ale jiným tělem.

Rozhraní

Rozhraním objektu se myslí to, jak je objekt viditelný zvenku.

Rozhraní objektu tvoří public metody, je to způsob, jakým s určitým typem objektu se může komunikovat.

Třídy implementující rozhraní musí povinně implementovat jeho metody, všechny.

Typy tříd

C#

Normální – prostě třída

Abstract

nemůže z ní být vytvořena instance, je používána pouze k dědění.

Je základem návrhového vzoru Decorator pattern.

Metody v ní nemůžou být private.

Umožňuje abstract metody, které slouží k přepsání odděděnou třídou.

Partial

Třída může být upravována jinými částmi programu.

Pokud někde bude napsána nová metoda v nové partial class, které existuje, tak do objektů tříd bude metody přídána.

Nemůže být děděna

Sealed

Tato třída nemůže být děděna.

Sealed se vztahuje i na Atributy a metody, které nemohou být děděny.

Static

Nelže vytvořit její instanci, nemůže být vytvořen její objekt.

Ve třídě mohou být pouze statické metody a atributy.

Nelze jí dědit.

Její metody a atributy se volají pomocí názvu třídy.

Vztahy objektů

Asociace

Asociace určuje základní vztah mezi dvěma entitami.

Mohou existovat nezávisle na sobě.

Zakreslujeme ji jako jednoduchou plnou čáru.

Jako výchozí je směr na obě strany, tedy že první entita má odkaz na druhou, a naopak druhá na první.

To lze změnit na to, že pouze jedna entita uchovává pouze odkaz na druhou. To je zakresleno přidáním šipky směrem na druhou entitu.

Agregace

Agregace reprezentuje vztah typu celek - část.

Zakreslujeme ji jako plnou čáru, zakončenou na jedné straně prázdným kosočtvercem. Ten je umístěn u té entity, která reprezentuje celek.

Z hlediska implementace je to tak entita, která drží kolekci prvků. Entita reprezentující část může existovat sama o sobě a být součástí i jiných kolekcí.

Kompozice

Kompozice je podobná agregaci, ale reprezentuje silnější vztah.

Entita části nemá bez celku smysl. Pokud zanikne celek, zanikají automaticky i jeho části.

Kompozici zakreslujeme stejně jako agregaci, kosočtverec je ovšem plný.

U entity reprezentující celek musí být multiplicita vždy 1.

Generalizace

Jedna entita dědí vlastnosti a chování jiné, prostě dědění

Je zakreslena čárou s prázdnou uzavřenou šipkou na konci třídy, od které dědí.

15 Programovací jazyky – vlastnosti, srovnání, popis způsobů tvorby i běhu programů

Co je programovací jazyk?

Programovací jazyky jsou jazyky pro tvorbu programů. Programem myslíme sadu instrukcí pro počítač, jak vyřešit nějaký problém – algorytmizujeme problémy.

Každý jazyk má určité diferencei, jak se liší od ostatních jazycích.Také je zde popularita, čitelnost, kompatibility. Hlavní vlastnost je míra abstrakce. Rozdělujeme je na vyšší a nižší.

Nižší

Jsou mnohem primitivnější, nemají komplexní syntaxi, píšeme podobný způsob jako píšeme rovnou příkazy pro procesor.

Procesor dělá určitou sadu instrukcí. Takže kód, který napíšeme, se nejdříve zpracuje do hexadecimálního kódu – strojový kód a poté do binárního. Tyto binární informace jsou poslány do procesoru.

Příklad: Jazyk Symbolických Adres - Assembler

Vyšší

Více zkušeností programátora, potřebují znalostu CPU, hardware.

Díky tomu, že nepracujeme s primitivním kódem jako v nižších, můžeme dělat složitější informace. Počítač vlastně neví, co je for each, procesor ho nezvládne. Kompilátor tedy zkompiluje kód do instrukcí.

Příklad: Java, Python, C#

Dělení dle kompilace

Kompilované jazyky

Příkladem kompilovaných jazyků je například C, C++. Výsledkem kompilací jsou hexa soubory – strojový kód. Jsou poměrně dost rychlé. Pokud kód obsahuje chybu, tak kompilátor hodí chybu. Díky tomu můžeme zjistit, kde je chyba před tím, než začneme spouštět kód. Nevýhoda je ale to, že zkompilovaný program je závislý na platformě a na typu procesoru nebo operačního systému. Kompilace občas může zabrat spoustu času v případě komplexního kódu. Jakmile máme zkompilovaný program, pokdu chceme udělat nějakou úpravu, tak musíme celou kompilaci udělat znovu.

Interpretované jazyky

Kompiluje jenom určité části a kompiluje během toho, jak jsou potřeba. Jedná se o tlumočníka. Člověk mluví anglicky, on to překládá do angličtiny.

Příklad je PHP.

Výhoda interpretovaných je to, že oproti kompilovaným je přenositelný mezi platformami. Jde o to, že interpreter je tlumočník. Proto je potřeba stáhnout v případě PHP exe soubor a tlumočníky, které dostanou na vstup

Není potřeba třeba dávat i datové typy, protože on je interpretuje postupně a můžou se měnit.

Jazyky s virtuálním strojem

Třeba Java – místo toho, aby on vytvářel rovnou hexadecimální – strojový kód, tak vytváří mezi binární soubor. V případě Javy to je bytecode. Řeší to problémy s hledáním chyb. Zde je zpracování jednodušší a rychlejší, interpretor to rychleji zpracuje.

Paradigmata

Paradigma zančí základní programovací styl – Alfa a omega toho, jak to má vypadat.

Dělá se na procedurální / imperatuvní a neprocedurální / deklarativní

Procedurální mají dva poddruhy – OOP a strukturované. Strukturovaný řeší problém pomocí funkce.

Neprocedurální jsou k tomu, abychom neřešili přesný postup, ale čeho má program dosáhnout, jakého cíle –

funkcionální a logický.

Funkcionální je program vytvořený z matematických funkcích - LIPS

Logické, ten řeší pomocí matematiky a logických výroků to, jak to vypadat.

Dělení podle účelu

Není žádný jazyk, který by dokázal řešit všechny problémy a nebyl by naprosto unvierzální.

Díky tomu se třeba Python zaměřuje na umělou inteligenci, analytiku a vědeckou činnost, Java zase se využívá na bankovní systémy, assembler je vhodný pro programování k hardware, C vhodný pro práci se stroji jako třeba Raspberry PI atd.

Statické a dynamické typování

U statických je pevně definován datový typ při zakládání. Na začátku je jasné, jaký datový typ může proměnná mít a nemůžeme ho měnit. Při špatném datovém typu spadne program.

Příklad: C# a Java

Dynamické typování – Proměnná se může měnit. Datový typ je tedy určen hodnotou jako takovou. Hodnoty vznikají během programu a nevýhoda je, že se nedá během kompilace zjistit, kde je chyba.

Syntaxe

Jazyky mají jinou syntaxi, Python třeba řeší odsazení místo složených závor při dělání kódu v bloku jako v ostatních jazycích. Poté třeba dědičnost, Python se píše do závorek vedle třídy, v C a C# se dávají vedle třídy po dvojtečce.